DAMG 6210 PROJECT

PHASE 1

TEAM MEMBERS

MEMBER 1 – Arvind Ranganath Raghuraman (002369728)

MEMBER 2 – Pooja Doddannavar (002059406)

Hospital Management System

This project aims to create a Relational DB for a hospital to ensure that hospitals can manage their transactions effectively without any hindrance due to data issues.

Problem Statement

It is essential that in a highly sensitive environment as hospital, the data is stored and retrieved properly without any issues. Moreover, there shouldn't be any issues related to data such as redundancy, duplicate records etc as it could have serious complications such as improper treatments,dosage etc and in worst case death of a patient. To address this, a Relational DB would help in the cause and it addresses the following

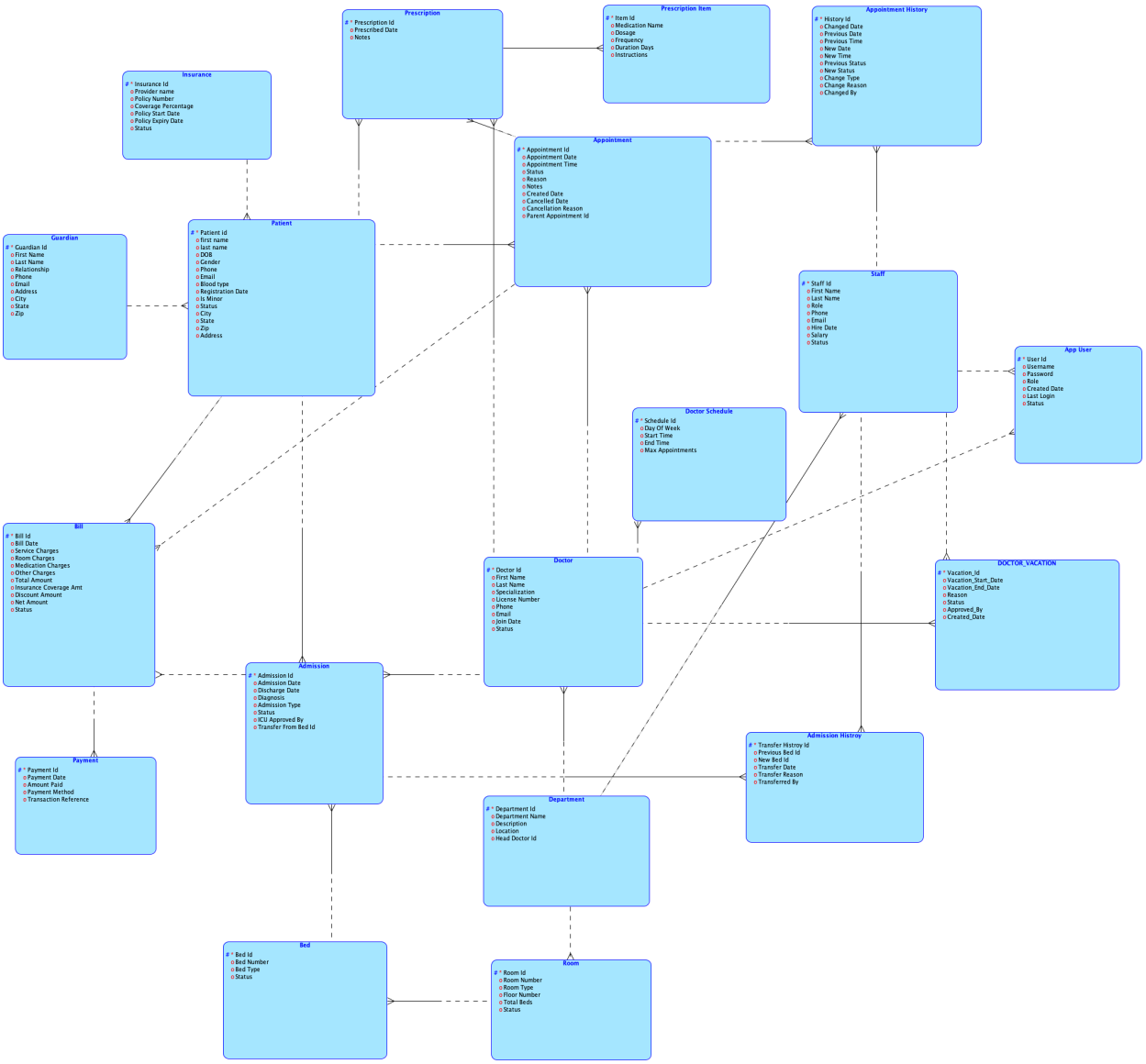
1) Manages Patients, Doctors and Appointments in Separate Modules.
2) Enforces business rules through constraints
3) Supports role-based access
4) Provides Reports so that the higher officials are aligned with day to day happenings in the hospital.

Logical Model Design

The following are the entities that encompass the entire modules

Entity	Description
Patient	Stores Patient Records
Guardian	Stores the Guardian Records (Note only if patient is less than 18)
Insurance	Stores the Insurance Related Details
Doctor Schedule	Stores weekly working hours and max appointments per day per doctor
Doctor Vacation	Stores the Doctor vacation dates and others
Department	Stores the Department Details and it contains the head doctor for that department

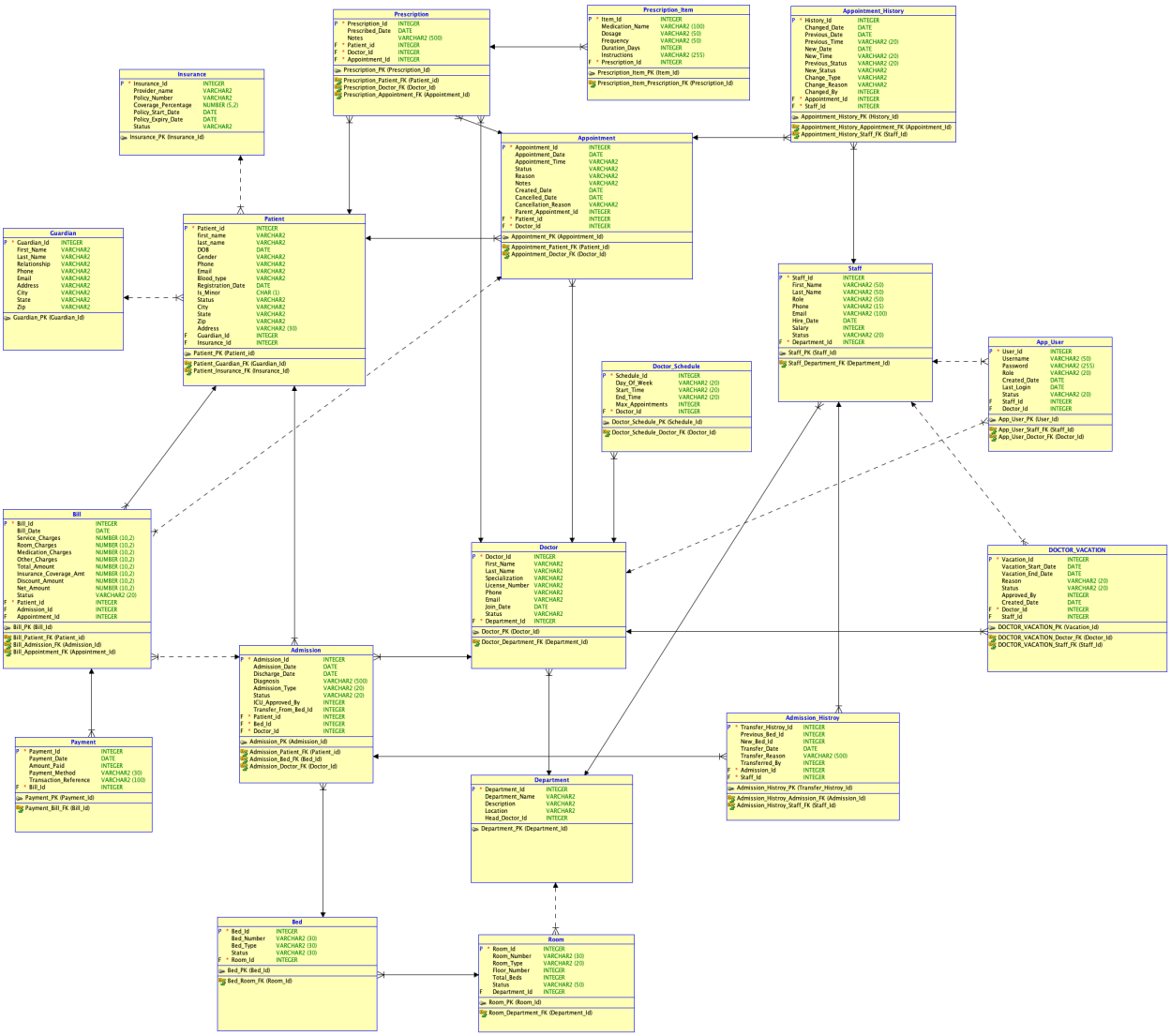
Staff	Stores the Staff Level Details, mainly the support persons
App_User	System Login Accounts linked to staff
Appointment	Tracks Booking, reschedulements and others
Appointment_History	Audit Trail for appointment changes
Prescription	Links a prescription to appointment
Prescription_Item	Individual Medication Item
Admission	Links a Patient to Admission
Admission_History	Audit Trails for Bed Transfers
Room	Defines the room number, type and occupancy status
Bed	Defines Individual Bed within the rooms
Bill	Generates changes with insurance and discount
Payment	Records payment against a bill



Key Relationships

- One **Patient** can have many **Appointments** (1:M)
- One **Doctor** can have many **Appointments** (1:M)
- One **Doctor** belongs to one **Department** (M:1)
- One **Appointment** can have one **Prescription** (1:1)
- One **Prescription** can have many **Prescription_Items** (1:M)
- One **Patient** can have many **Admissions** (1:M) — but only one active at a time
- One **Bed** can be assigned to one active **Admission** (1:1 active)
- One **Room** can have many **Beds** (1:M)
- One **Bill** is generated per Admission or Appointment (1:1)
- One **Bill** can have many **Payments** (1:M)
- One **Staff** member has one **App_User** login (1:1)
- One **Patient** may have one **Guardian** if minor (1:1)
- One **Patient** may have one **Insurance** policy (M:1)

Relational Model



Solution Approach

Conceptual Model – Identify the entities and attributes

Logical Model – Convert the conceptual model into a relational schema with primary and foreign keys

Normalization – Apply Normalization rules to eliminate redundancy and anomalies.

Implementation – Create Required Triggers, Stored Procedures and others in Oracle Autonomous DB.

Testing – Validate data integrity, performance and relationships.

Normalization Steps

1NF – Each Column should have atomic value, no repeating groups

Address stores the house or street name while city, zip and
state stores the city, state and zip code attributes respectively in patient and guardian table.

Prescription_Item was created so that no multiple medications can be stored in prescription table

Doctor Schedule was created to avoid redundant schedule values.

Appointment History captures each status change as sep row.

2NF – Must be in 1NF. All Non key attributes must depend on entire primary key (No Partial Dependencies)

1. **Prescription** table initially had Doctor_Name and Patient_Name stored alongside Doctor_Id and Patient_Id. These attributes depend only on Doctor_Id or Patient_Id individually, not on Prescription_Id. They were removed; names are retrieved via JOIN to Doctor and Patient tables.

2. **Bill** table initially included Doctor_Name and Department_Name for reporting convenience. These depend on Appointment_Id or Admission_Id, not on Bill_Id. They were removed; data is accessed via join.

3. **Admission_History** uses a surrogate primary key (Transfer_History_Id). All attributes (Previous_Bed_Id, New_Bed_Id, Transfer_Date) fully depend on Transfer_History_Id — no partial dependency exists.

4. **Appointment_History** uses surrogate key History_Id. All change-tracking attributes depend entirely on History_Id — no partial dependencies.

5. **Prescription_Item** uses Item_Id as PK. Medication_Name, Dosage, and Frequency fully depend on Item_Id, not on Prescription_Id alone.

3NF – Must be in 2NF. There shouldn't be any transitive dependencies ie a non key attribute must not depend on another non key attribute

Actions Taken:

- **Patient → Insurance transitive dependency removed:** Originally, Coverage_Percentage, Provider_Name, and Policy_Number were stored on the Patient table. These attributes describe the Insurance policy, not the Patient. They were extracted into a standalone **Insurance** table. Patient references Insurance via Insurance_Id (FK).

- **Room → Department transitive dependency resolved:** Room has Department_Id as FK. Department_Name and Location are stored in the Department table, not repeated in Room. Accessing Department_Name from a Room query requires a JOIN — not a column on Room.

- **Staff → Department transitive dependency resolved:** Department_Name was initially considered as a column on Staff for quick lookup. Removed — Staff references Department via Department_Id FK; name is retrieved via JOIN.

- **Doctor → Department transitive dependency resolved:** Similarly, Department_Name and Location are not stored on Doctor. Doctor references Department via FK.

- **Bill → Insurance transitive dependency resolved:** Insurance_Coverage_Amt is calculated at billing time using the Coverage_Percentage from Insurance. The coverage percentage itself is not stored on Bill — it is applied as a computed value and the resulting amount stored. This prevents a transitive dependency where Coverage_Percentage (an insurance attribute) would otherwise appear on the Bill table.
- **App_User → Staff transitive dependency resolved:** App_User links to Staff via Staff_Id. Role, department, and contact info remain on Staff — not duplicated on App_User.

Further Steps

We will be working on Patient Module and the following are the deliverables we aim to deliver

1) DDL Scripts (For Patient, Guardian and Insurance Tables) with primary keys, constraints enforced
2) Stored Procedures for Insert, Updating and Deleting records. Triggers to prevent duplicate records and enforce guardian requirements.
3) Sample views to view patient/insurance reports.
4) Prepare tests to validate business rules.